



© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2023

M. Frisbie, *Building Browser Extensions*

https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-8725-5_8

8. Popup and Options Pages

Matt Frisbie¹

(1) California, CA, USA

Browser extensions almost always need to allow the user to control some aspect of its behavior via a user interface. The popup and options pages are the bread-and-butter extension components that should be used for this need. Content scripts can also be used to offer an in-page user interface, but there are important tradeoffs and complications when doing so.

Note Refer to the *Content Scripts* chapter for coverage on these tradeoffs.

Popup Pages

As its name suggests, the popup page is a web page that renders inside a container that “pops up” over the browser’s web page. It is completely controlled by the browser extension and guaranteed to overlay the browser’s current web page (Figure [8-1](#)).

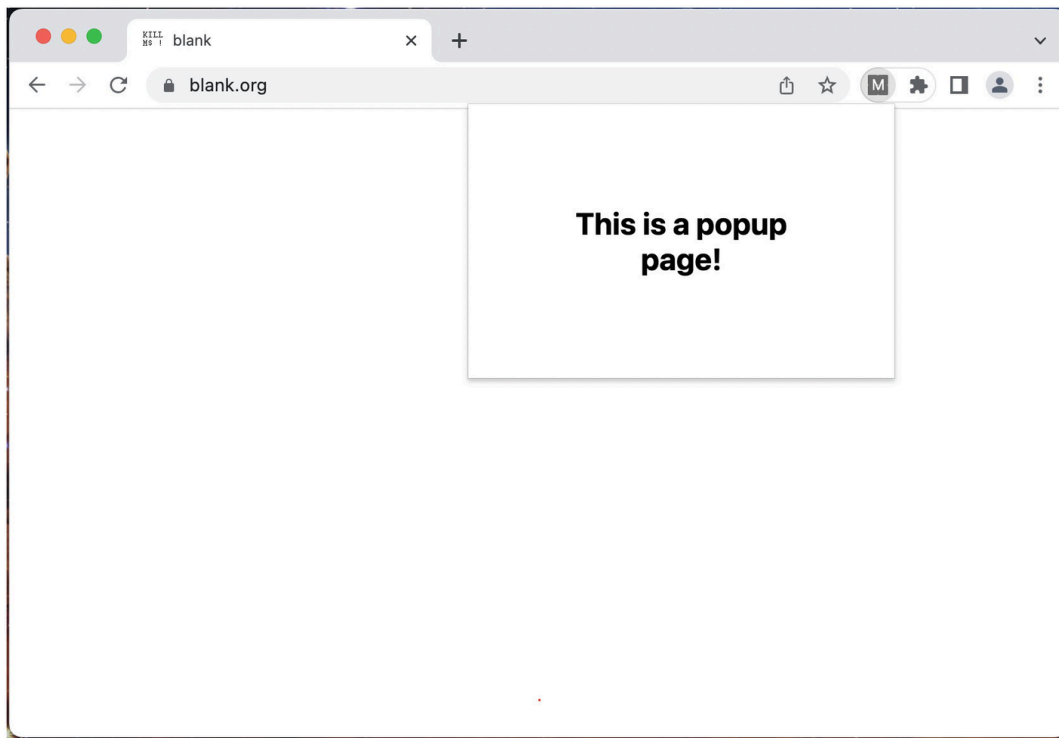


Figure 8-1 A simple popup page

Popup Page Properties

The popup page container can be thought of as a simple browser tab without any browser chrome: no URL bar, no browser controls, etc. In nearly every other respect, the popup page is like any other web page. The following demo extension demonstrates some basic properties of popup pages (Figure [8-2](#)):

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "action": {
    "default_popup": "popup.html"
  }
}
```

Example 8-1a manifest.json

```
<!DOCTYPE html>
<html>
  <head>
    <link href="popup.css" rel="stylesheet" />
  </head>
```

```
<body>
  <h1>This is a popup page!</h1>
  <div id="url"></div>
  <div id="xid"></div>
  <a href="popup.js">popup.js</a>
  <script src="popup.js"></script>
</body>
</html>
```

Example 8-1b popup.html

```
document.querySelector("#url").innerHTML = `
<pre>Page URL: ${window.location.href}</pre>
`;
document.querySelector("#xid").innerHTML = `
<pre>Extension ID: ${chrome.runtime.id}</pre>
`;
```

Example 8-1c popup.js

```
body {
  text-align: center;
  min-width: 20rem;
  padding: 4rem;
}
```

Example 8-1d popup.css

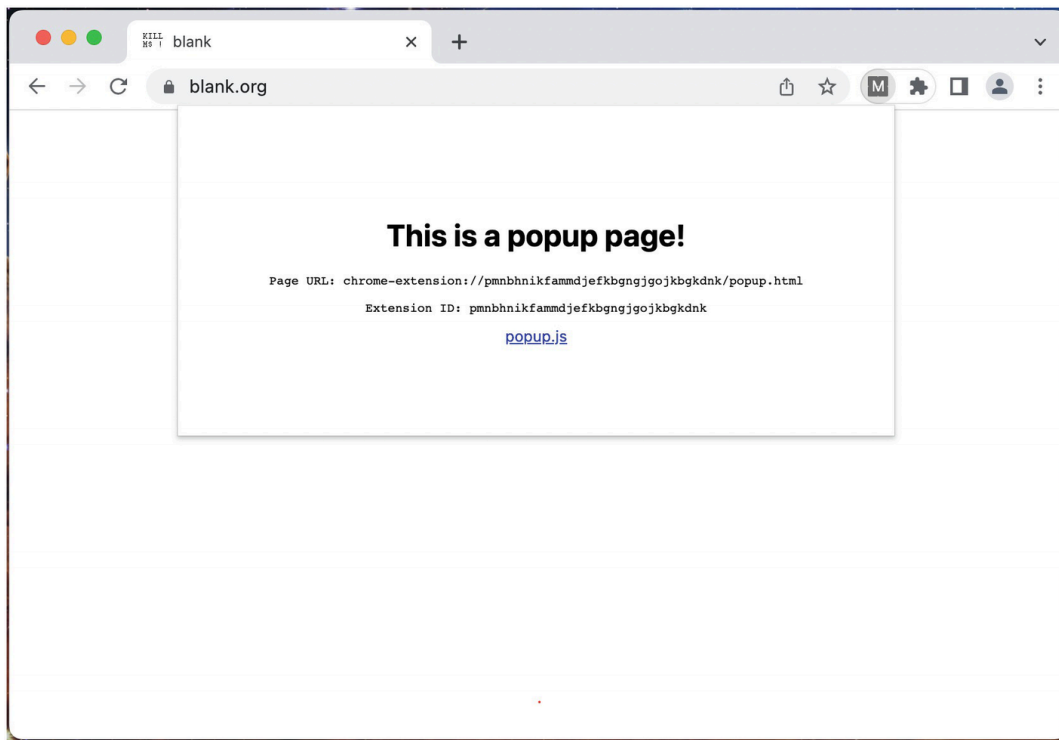


Figure 8-2 The opened popup page of the demo extension

In `manifest.json`, the popup page is declared by specifying the path to the popup HTML file via the `action.default_popup` value. As described in the *Extension Manifest* chapter, the `action` dictionary defines how the toolbar icon button appears and behaves:

- If the `action` dictionary is defined with a `default_popup` value, clicking that button will open the popup.
- If the `action` dictionary is defined *without* a `default_popup` value, clicking the toolbar icon button will instead call the `chrome.action.onClicked()` handler wherever it is defined.

The popup container will be sized to fit the content of the popup page, but browsers will handle the sizing rules slightly differently. For example, Google Chrome sets no upper bound on the maximum size of the popup, allowing it to even expand larger than the browser window itself. Conversely, Mozilla Firefox limits the popup to be no larger than 800px by 600px.

Tip To handle these variable popup size limits, you should either make the popup page responsive or limit the amount of content that appears inside the popup.

Because the popup page is served from the browser's extension protocol – in Google Chrome, `chrome-extension://` – the page's JavaScript has access to the WebExtensions API. The popup page is not restricted from navigating to other URLs, so clicking the link to `popup.js` will successfully navigate the popup page to a new URL.

More advanced extensions will often build multi-view single page applications inside the popup page. These extensions should use the HTML page that the manifest specifies as a popup as the equivalent of a web page's `index.html` entrypoint.

Opening and Closing Popup Pages

Popup pages are intended to show only after an explicit user action. Browsers intentionally prevent you from programmatically opening a popup page to prevent extensions from spamming the user with page overlays. Therefore, there are essentially two ways of opening the popup page:

- Clicking the toolbar icon button when the manifest has `action.default_popup` defined
- Entering the key shortcut defined in the manifest's `commands._execute_action` when `action.default_popup` is defined

Note There is a `chrome.action.openPopup()` method listed in extension documentation, but at the time this book was written this method is still not supported. When support is eventually rolled out, it will still only be usable directly after an eligible user interaction; therefore, programmatic popup page opening will remain effectively disallowed.

Once the popup is open, there are many more ways to trigger a close. From popup page's perspective, closing the popup is identical to a tab being closed. The following actions will trigger a popup close:

- Clicking the toolbar icon button when the popup is already open

- Clicking outside the opened popup window *and* inside the same browser window
- Calling `window.close()` from inside the popup JavaScript
- Using the `escape` key

Note Importantly, clicks outside the browser window will *not* close the popup. This behavior allows you to open one popup page per window. It also means must assume the user can have multiple popup pages open simultaneously.

Changing the Popup Page

You are able to change the popup page via the WebExtensions API. Any changes made will persist through browser sessions. The following snippet demonstrates how to change the popup page:

```
chrome.action.setPopup({  
  popup: chrome.runtime.getURL("new-popup.html")  
});
```

Detecting Popup State

Browser extensions implicitly keep track of active window objects that are controlled by the extension. The `chrome.extension.getViews()` method provides access to this list, and you are able to provide a `type` to filter only the popup views, shown here:

```
// Returns an array of window objects  
chrome.extension.getViews({ type: "popup" });
```

If you wish to detect if a view is being rendered inside a popup, you can perform a simple check against the `window` object:

```
const popups = chrome.extension.getViews({ type: "popup" });  
console.log(`Inside a popup: ${popups.includes(window)}`);
```

Suggested Use

When developing popup pages, you should follow these strategies:

- Popup pages are best used for user interfaces that the user needs to access quickly without losing the context of the current page that they are on.
- As some browsers will restrict the page dimensions of the popup page, a good rule of thumb is that the interface should contain approximately as much content as a phone app.
- Because popup pages will be closed and re-opened quickly, the popup page should efficiently render and not rely on long-running operations that may need to terminate early.
- When you need the popup page be stateful or load data, it should aggressively utilize a caching strategy to avoid any UX slowdowns.
- In situations where you need more screen real estate, a popup page can offer links or buttons that will open the options page or other views in a separate browser tab.

Options Pages

The options page is a web page that renders in one of two ways: as a standalone tab, or inside a modal container over the browser's extension manager page (Figures [8-3](#) and [8-4](#)). It is completely controlled by the browser extension.

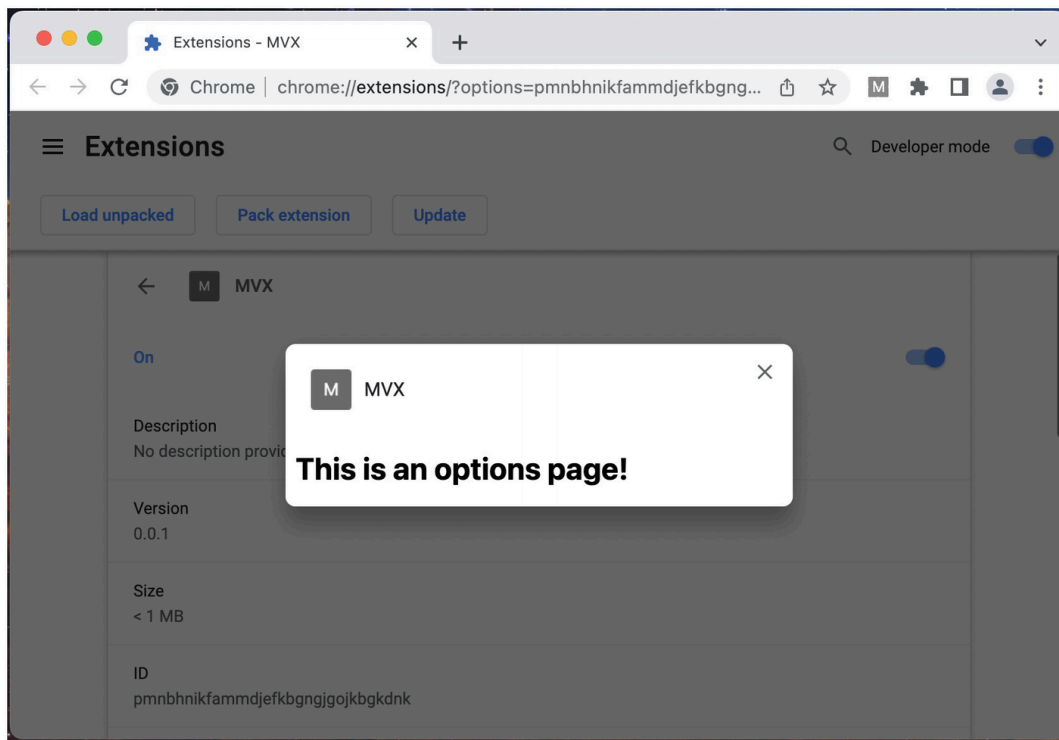


Figure 8-3 The options page rendered inside the extension manager page modal

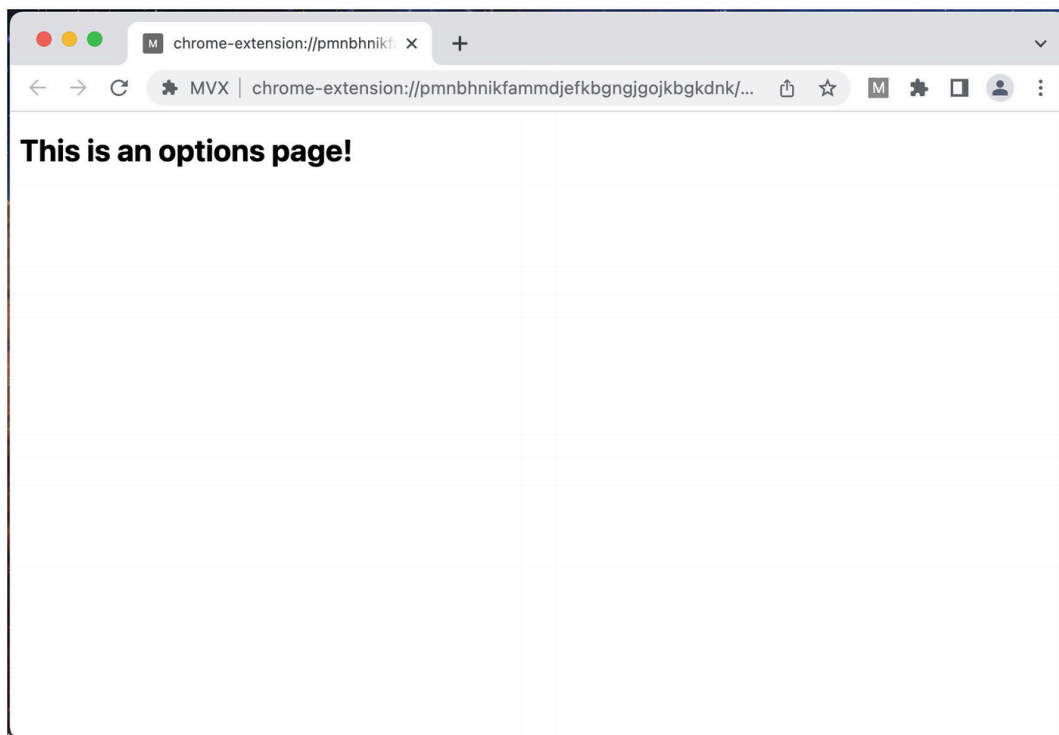


Figure 8-4 The options page rendered in a tab

Options Page Properties

When opened in a tab, the options page is like any other web page. When opened as a modal, the options modal container can be thought of as a simple browser tab without any browser chrome: no URL bar, no browser controls,

etc. The following demo extension demonstrates some basic properties of options pages:

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "action": {
    "default_popup": "popup.html"
  },
  "options_ui": {
    "open_in_tab": false,
    "page": "options.html"
  }
}
```

Example 8-2a manifest.json

```
<!DOCTYPE html>
<html>
  <body>
    <h1>Popup</h1>
    <button id="opts">Options</button>
    <a href="options.html" target="_blank">options.html</a>
    <script src="popup.js"></script>
  </body>
</html>
```

Example 8-2b popup.html

```
document.querySelector("#opts").addEventListener(
  "click",
  () => chrome.runtime.openOptionsPage());
```

Example 8-2c popup.js

```
<!DOCTYPE html>
<html>
  <body>
    <h1>Options</h1>
```

```
<a href="options.js">options.js</a>
</body>
</html>
```

Example 8-2d options.html

In `manifest.json`, the options page is declared by specifying the path to the options HTML file via the `options_ui.page` value. The options page default rendering can occur in one of two ways:

- If `options_ui.open_in_tab` is `false`, the options page by default will open in a browser tab.
- If `options_ui.open_in_tab` is `true`, the options page by default will open in a modal over the browser's extension manager page.

If opened as a modal, the options container will be sized to fit the content of the options page. The options page can specify body dimensions to grow or shrink as needed.

Because the options page is served from the browser's extension protocol – in Google Chrome, `chrome-extension://` – the page's JavaScript has access to the WebExtensions API. In both tab and modal mode, the options page is not restricted from navigating to other URLs, so clicking the link to `options.js` will successfully navigate the options page to a new URL.

More advanced extensions will often build multi-view single page applications inside the options page. These extensions should use the HTML page that the manifest specifies as options as the equivalent of a web page's `index.html` entrypoint.

Opening and Closing Options Pages

There are several ways to open the options page, and unlike the popup, the options page *can* be opened programmatically. The following actions will open the options page, and will always respect the `options_ui.open_in_tab` value:

- Right clicking the toolbar icon button and selecting *Options*
- Call `chrome.runtime.openOptionsPage()`

When the options page is opened in a tab, it closes like a standard web page. When opened as a modal, the following actions will trigger an options close:

- Clicking the X button inside the options page modal
- Calling `window.close()` from inside the options JavaScript
- Using the `escape` key when viewing the options modal
- Closing the extension manager tab

NoteThe “options page” designation refers mainly to the stems from the browser’s ability to access the page via context menu item or WebExtensions API method. Once opened as a tab, the options page is no different than any other extension HTML file opened via the extension protocol URL.

Detecting Options Pages

When options pages are opened as a tab, they are treated as any other tab and therefore can be detected via `chrome.extension.getViews()` or `tabs.query()`. However, when opened as a modal, the options page will have the following limitations:

- `chrome.tabs.query()` will not return the options page
- The `chrome.tabs.onCreated` and `chrome.tabs.onUpdated` events will not fire when the modal is opened or changes
- The modal options page cannot be sent messages via `chrome.tabs.connect()` or `chrome.tabs.sendMessage()`
- `chrome.extension.getViews({ type: "tab" })` will not return the modal options window. Note that `chrome.extension.getViews()` will still include the modal options window.

Suggested Use

When developing options pages, you should follow these strategies:

- Options pages are best used for user interfaces that contain controls for how the extension behaves.
- The options page has no restrictions on content. However, since a very common path to accessing it is to right click on the toolbar icon and select “Options,” the options page should probably resemble a page with which to control extension options.
- There isn’t much of an upside to using the modal version of the extension page. Users are likely not familiar with the extension management interface, so displaying a modal view over it could be confusing. I recommend sticking to using the tab version.

Content Script Restrictions

One tricky and important behavior to note is that programmatic access to options pages (and any other extension URLs) is forbidden. Consider the following example extension, which has a popup page with four buttons, each of which attempts to open the options page in a different way:

```
{
  "name": "MVX",
  "version": "0.0.1",
  "manifest_version": 3,
  "action": {
    "default_popup": "popup.html"
  },
  "options_ui": {
    "open_in_tab": true,
    "page": "options.html"
  },
  "permissions": ["scripting"],
  "host_permissions": ["<all_urls>"]
}
```

Example 8-3a manifest.json

```
<!DOCTYPE html>
```

```
<html>
  <body>
    <h1>Popup</h1>
    <button id="popup-api">Popup API</button>
    <button id="popup-url">Popup URL</button>
    <button id="cs-api">Content Script API</button>
    <button id="cs-url">Content Script URL</button>
    <script src="popup.js"></script>
  </body>
</html>
```

Example 8-3b popup.html

```
function openOptionsWithUrl() {
  window.open(chrome.runtime.getURL("options.html"));
}
function openOptionsWithApi() {
  chrome.runtime.openOptionsPage();
}
async function sendFnToActiveTab(fn) {
  let [tab] = await chrome.tabs.query({
    active: true,
    currentWindow: true,
  });
  chrome.scripting.executeScript({
    target: { tabId: tab.id },
    function: fn,
  });
}
document.querySelector("#popup-api").addEventListener(
  "click",
  () => openOptionsWithApi());
document.querySelector("#popup-url").addEventListener(
  "click",
  () => openOptionsWithUrl());
document.querySelector("#cs-api").addEventListener(
  "click",
  () => sendFnToActiveTab(openOptionsWithApi));
document.querySelector("#cs-url").addEventListener(
  "click", () => sendFnToActiveTab(openOptionsWithUrl));
```

Example 8-3c popup.js

```
<!DOCTYPE html>
<html>
  <body>
    <h1>Options</h1>
  </body>
</html>
```

Example 8-3d options.html

Note The content script buttons must be used with a valid active tab such as <https://blank.org>.

The popup page's four buttons each offer one of the four combinations of popup origin or content script origin, and API open or URL open. Initially, nothing here should appear out of the order. When you click the two popup buttons, the options page will correctly open without incident.

However, after clicking each of the content script buttons, the options page will not correctly open and you will see the errors shown in Figures [8-5](#) and [8-6](#).

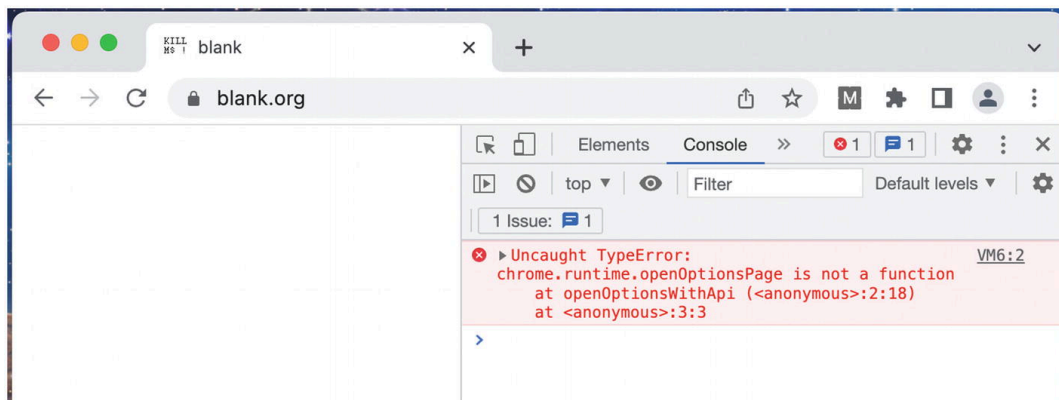


Figure 8-5 The error shown after clicking the Content Script API button

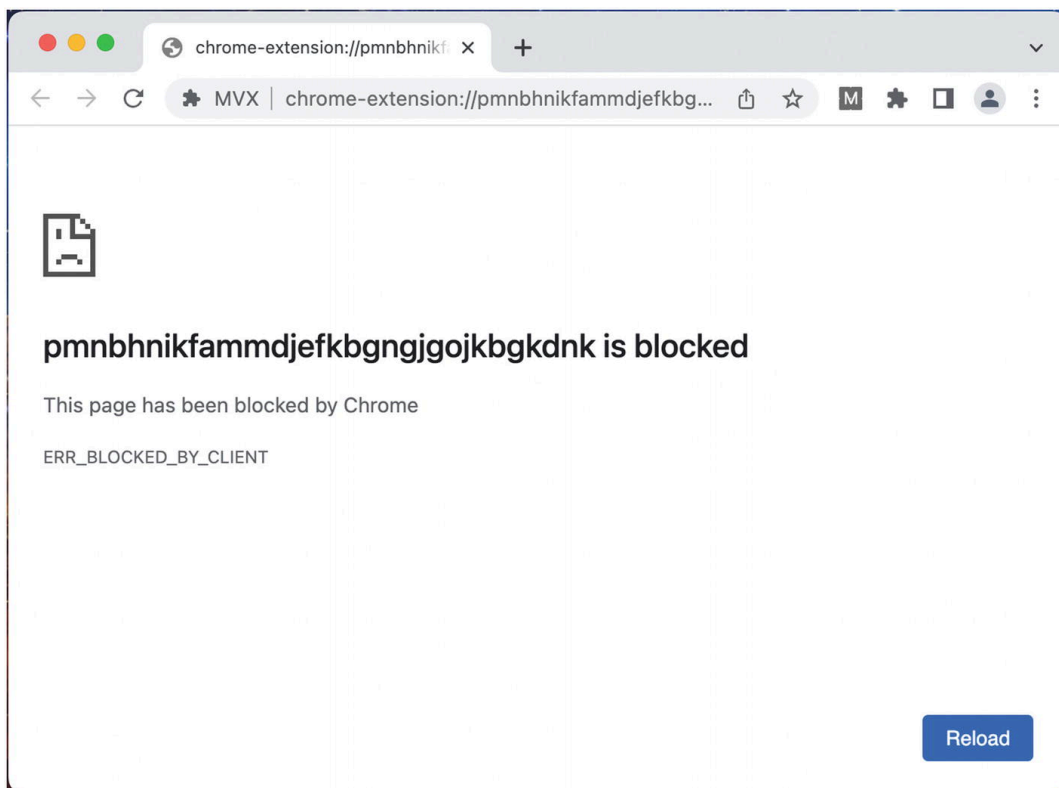


Figure 8-6 The error shown after clicking the Content Script URL button

The browser prevents the content script from programmatically opening the options page. In the content script, the API method is simply removed, and the extension URL is prevented from accessing the extension's filesystem.

The reason for this is presumably security related. The potential for content scripts to abuse their ability to kick the user into a new tab is too great, so content scripts are restricted from doing so.

Summary

In this chapter, you learned about the two major building user interface building blocks of browser extensions: popup pages and options pages. The chapter took you through the details of how each one operates inside the browser, how they can fit into the user flow, and the advantages and disadvantages of using each one.

In the next chapter, you will learn about how content scripts can complement and enhance popup and options pages, as well as enable an

entirely new extension domain in which to architect user interfaces.